Primer: Neural Network Efficient Estimator (NNES)

econirl package documentation

1 Problem and Motivation

Sieve methods like SEES (**?**) approximate the value function with deterministic basis functions. This works when the value function is smooth and low-dimensional, but the number of basis terms needed grows exponentially in the state dimension. Neural networks are universal approximators that do not suffer from this curse of dimensionality, but plugging a neural network into a structural estimation framework creates a new problem: the approximation error in the value function contaminates the parameter estimates and invalidates the standard errors.

? introduces the Neural Network Efficient Estimator (NNES), which solves this problem by embedding the neural value function approximation inside the Nested Pseudo-Likelihood (NPL) framework of **?**. The critical insight is that the NPL policy-iteration operator has a *zero Jacobian property*: at the true conditional choice probabilities (CCPs), the derivative of the operator with respect to the value function vanishes. This means first-order approximation errors in the neural V-network drop out of the likelihood score, producing *Neyman orthogonality*. The estimator is $\sqrt{n}$-consistent and achieves the semiparametric efficiency bound without bias correction or cross-fitting.

2 Model and Notation

Table 1: Common DDC notation shared across all primers.

Symbol	Name	Definition
$s \in \mathcal{S}$	State	Discrete or continuous
$a \in \mathcal{A}$	Action	Finite: $\lvert\mathcal{A}\rvert = J$
s'	Next state	Drawn from $P(s' \mid s, a)$
$\beta \in [0, 1)$	Discount factor	
$\sigma > 0$	Scale parameter	Logit noise scale (= 1 by convention)
$\theta \in \Theta \subset \mathbb{R}^{d_\theta}$	Structural parameters	Unknown
$\phi(s, a)$	Feature vector	Known functions of (s, a), dimension K
$u(s, a; \theta)$	Flow utility	$\theta^\top \phi(s, a)$ (linear-in-parameters)
$P(s' \mid s, a)$	Transition kernel	Known or estimated from data
$Q(s, a)$	Choice-specific value	$u(s, a; \theta) + \beta \sum_{s'} P(s'\lvert s, a)\, V(s')$
$V(s)$	Integrated value	$\sigma \log \sum_a \exp(Q(s, a)/\sigma)$
$\pi(a \mid s)$	Choice probability	$\exp\big((Q(s, a) - V(s))/\sigma\big)$
τ	Trajectory	$(s_0, a_0, s_1, a_1, \ldots)$ from one agent
$\mathcal{D}$	Panel data	Collection of N observed trajectories

The agent observes continuous state $x \in \mathcal{S} \subset \mathbb{R}^d$ and chooses discrete action $a \in \mathcal{A} = \{1, \ldots, J\}$ to maximize discounted expected utility with additive Type I extreme value shocks.

Symbol	Name	Definition
$e(a,x)$	Emax correction	$\gamma_{\text{Euler}} - \log P(a\|x)$ (Hotz-Miller)
$\phi_\theta[P]$	Evaluation operator	Value function implied by CCPs P at parameter θ
$\Lambda_\theta[V]$	Improvement operator	CCPs implied by value function V at parameter θ
$\Psi_\theta = \Lambda_\theta \circ \phi_\theta$	Policy iteration	Fixed point: $P^* = \Psi_{\theta_0}(P^*)$
$V_\gamma(x)$	Neural V-network	$G(x;\gamma) - G(x_0;\gamma)$ (anchored at x_0)
ω_n	Bellman penalty	Enforces $V_\gamma \approx \phi_\theta[P]$

The Hotz-Miller inversion expresses the value function as a linear operator on the CCPs. Given any admissible CCP vector P, the evaluation operator solves

$$\phi_\theta[P](x) = (I - \beta T_P)^{-1}\big(\textstyle\sum_a P(a|x)\, u_a(x;\theta) + \sigma \sum_a P(a|x)\, e(a,x)\big), \tag{1}$$

where T_P is the policy-weighted transition operator $(T_P f)(x) = \sum_a P(a|x)\mathbb{E}[f(x')|x,a]$ and $e(a,x) = \gamma_{\text{Euler}} - \log P(a|x)$ is the Hotz-Miller emax correction. The improvement operator then recovers CCPs via the logit formula $\Lambda_\theta[V](a|x) = \exp(Q_a/\sigma)/\sum_{a'} \exp(Q_{a'}/\sigma)$.

The neural V-network is anchored at a fixed state x_0 via $V_\gamma(x) = G(x;\gamma) - G(x_0;\gamma)$, which enforces $V_\gamma(x_0) = 0$ for all γ. This removes the unidentified level of the value function and stabilizes optimization.

3 Core Results

Proposition 1 (Zero Jacobian). Under additive separability and conditional independence (Assumptions 1–2), the Jacobian of the policy-iteration operator Ψ_θ with respect to P vanishes at the fixed point:

$$\left.\frac{\partial \Psi_{\theta_0}(P)}{\partial P}\right|_{P=P^*} = 0.$$

This extends Proposition 2 of **?** from finite to continuous state spaces. At the optimum, it is not possible to increase expected utility by changing choice probabilities, so the Jacobian is zero.

Proposition 3 (Value function orthogonality). The derivative of the evaluation operator with respect to the value function also vanishes at the fixed point:

$$\left.\frac{\partial \phi_{\theta_0}(P,V)}{\partial V}\right|_{V=V^*} = 0.$$

Any small first-stage error in approximating V affects the likelihood score at second order only.

Proposition 4 (Neyman Orthogonality). The Gâteaux derivative of the expected score with respect to V in any direction $(V - V^*)$ is zero:

$$\left.\frac{d}{dt}\,\mathbb{E}[\psi(W;\theta_0, V^* + t(V - V^*))]\right|_{t=0} = 0.$$

This means the NNES score function is automatically orthogonal to V-network estimation error, without requiring an explicit bias correction term (**?**).

Theorem 4.3 (Asymptotic Normality and Efficiency). If $\|V_\gamma - V^*\|_2 = o_p(n^{-1/4})$, the NNES estimator satisfies

$$\sqrt{n}(\hat{\theta} - \theta_0) \xrightarrow{d} N\big(0,\ I_{\theta_0}^{-1}\big),$$

achieving the semiparametric efficiency bound. The information matrix is *block-diagonal* between θ and the V-network parameters, so standard errors from the pseudo-likelihood Hessian are valid without the Schur complement marginalization that SEES requires.

Contrast with SEES. **?** show that SEES is also asymptotically efficient, but its information matrix is not block-diagonal. SEES standard errors require the Schur complement (Corollary 3.1), which involves inverting the full Hessian over (θ, α). NNES avoids this because the zero Jacobian makes the information matrix block-diagonal by construction.

4 Estimation Algorithm

Algorithm 1: NNES with NPL Bellman (**?**)

Input: Panel $\{(x_i, a_i)\}_{i=1}^N$, features $u_a(x;\theta)$, transitions $f(x'|x,a)$, discount β, network architecture (L, W), penalty ω_n, anchor x_0

`// Step 0: Initialize CCPs`

Estimate initial CCPs $\hat{P}_0(a|x)$ from data frequencies (tabular) or neural classifier (continuous)

for $k = 1, 2, \ldots, K$ *(outer iterations)* **do**

`// Phase 1: Train V-network on NPL target`

Compute emax corrections: $e(a,x) = \gamma_{\text{Euler}} - \log P_{k-1}(a|x)$

Compute policy-weighted transitions: $F_\pi[x,x'] = \sum_a P_{k-1}(a|x)\, f(x'|x,a)$

Solve NPL target: $W = (I - \beta F_\pi)^{-1}(E_{\text{flow}} + \sigma\, E_{\text{emax}})$

Train V_γ via supervised regression:

$\hat{\gamma} \leftarrow \arg\min_\gamma \frac{1}{M}\sum_{m=1}^M (V_\gamma(x_m) - W(x_m))^2 + \omega_n \cdot \rho_n(\gamma, \theta)$

where ρ_n is the Bellman residual penalty on a grid

Implied CCPs: $P_\gamma(a|x) = \Lambda_\theta[V_\gamma](a|x)$

`// Phase 2: Structural MLE`

Profile log-likelihood: $\ell(\theta) = \sum_{i=1}^N \log P_\gamma(a_i|x_i;\theta)$

$\hat{\theta} \leftarrow \arg\max_\theta \ell(\theta)$ via L-BFGS-B

`// Outer update`

Update CCPs from estimated policy: $P_k \leftarrow \Lambda_{\hat{\theta}}[V_{\hat{\gamma}}]$

end

`// Standard errors (valid by Neyman orthogonality)`

$H_\theta \leftarrow$ Hessian of $\ell(\hat{\theta})$ at the final $\hat{\theta}$

$\text{SE}(\hat{\theta}) \leftarrow \sqrt{\text{diag}(-H_\theta^{-1})}$

Output: $\hat{\theta}$, $V_{\hat{\gamma}}$, π^*, $\text{SE}(\hat{\theta})$, log-likelihood

Implementation notes. The V-network is a ReLU MLP with L hidden layers of width W (defaults: $L = 2$, $W = 32$). Anchoring enforces $V_\gamma(x_0) = 0$ by subtracting $G(x_0;\gamma)$ from all outputs. Phase 1 uses Adam with gradient clipping. Phase 2 uses L-BFGS-B on the pseudo-likelihood. The

number of outer iterations controls how many times Phases 1 and 2 alternate. The paper shows that one iteration is sufficient for $\sqrt{n}$-consistency, but finite-sample performance improves with additional iterations that progressively align the V-network with model-consistent CCPs.

NPL vs NFXP Bellman. The NPL variant (Algorithm **??**) uses the Hotz-Miller emax correction to construct the V-target. The NFXP variant instead trains V_γ to minimize the soft Bellman residual $\|V_\gamma - T_\sigma(V_\gamma;\theta)\|^2$. Only the NPL variant has the zero Jacobian property. The NFXP variant lacks Neyman orthogonality, so V-approximation errors feed directly into the score and standard errors are unreliable.

5 Results

Table 2: Rust bus engine (90 bins, $\beta = 0.9999$, 20,000 obs). NNES uses a 2-layer ReLU network with 32 hidden units.

	True	NFXP	NNES
$\hat{\theta}_c$	0.001	0.001468	0.022693
SE($\hat{\theta}_c$)	—	0.000312	0.003645
$\widehat{RC}$	3.0	3.0731	3.0488
SE($\widehat{RC}$)	—	0.0530	0.0473
Log-likelihood	—	−4193.1	−4193.3
Time (s)	—	11.6	20.7

We run NNES and NFXP on a Rust bus engine with 90 mileage bins, discount $\beta = 0.9999$, and 20,000 observations (true $\theta_c = 0.001$, $RC = 3.0$). NNES uses a 2-layer ReLU V-network with 32 hidden units, 500 V-network epochs per outer iteration, and 10 outer iterations.

The log-likelihoods are nearly identical (gap of 0.18 nats), confirming that NNES finds the same optimum as NFXP. The replacement cost is well-recovered by both estimators. The operating cost coefficient $\theta_c = 0.001$ is notoriously hard to identify in the Rust bus model because the flow utility gradient is small relative to the replacement cost. Neural structural estimators face an additional challenge: the V-network can partially absorb the linear operating cost term into the value function approximation, creating a tradeoff between θ_c and V_γ that the outer iterations gradually resolve.

Code. The companion script `nnes_run.py` in this folder generates Table **??**. The NNES estimator is implemented in `src/econirl/estimation/nnes.py`. To regenerate:

```
python papers/econirl_package/primers/nnes/nnes_run.py
```

6 Comparison to Other Estimators

	NFXP	CCP	SEES	NNES	TD-CCP
V-approximation	Exact	None	Sieve	Neural	Neural
Inner loop	VI/NK	None	None	SGD	SGD
Neyman orthogonal	N/A	Yes	No	Yes (NPL)	Yes
Block-diagonal $I(\theta)$	Yes	Yes	No	Yes	Yes
Requires $P(s'\|s,a)$	Yes	Yes	Yes	Yes	No
Scales to $d > 10$	No	No	Limited	Yes	Yes

Table 3: NNES in context. Only NPL-based methods (CCP, NNES, TD-CCP) have Neyman orthogonality.

NNES inherits the statistical elegance of NPL (zero Jacobian, block-diagonal information, automatic orthogonality) while using neural networks to overcome the curse of dimensionality. TD-CCP (**?**) achieves similar properties without requiring transition densities, but uses temporal-difference learning instead of supervised regression, which can be less stable.

The main practical limitation of NNES relative to NFXP is sensitivity to initialization and the number of outer iterations. NFXP solves the Bellman equation exactly, which pins down the value function as a deterministic function of θ. NNES must learn this mapping through alternating optimization, and insufficient iterations can leave the V-network on a different point of the identification manifold.